

Face Recognition Module — Implementation Plan

Overview

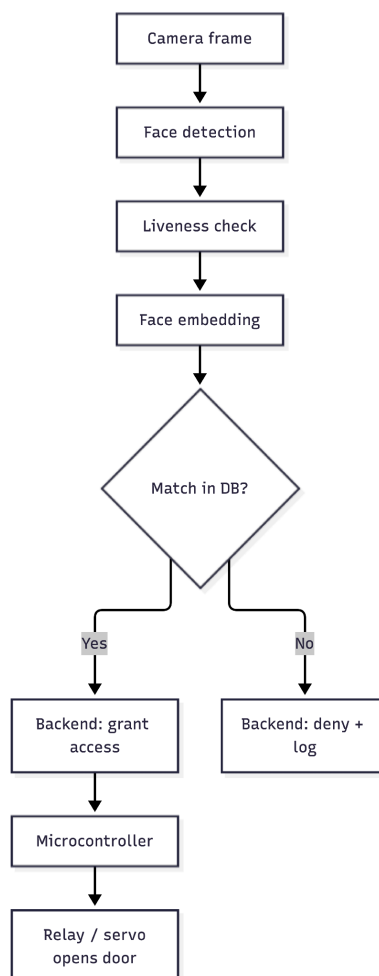
The face recognition module identifies a registered person from a camera image and tells the Python backend who they are, so the backend can decide whether to open the door.

The module covers four jobs:

1. scanning faces
2. enrolling (memorizing) new people
3. storing face data in a database
4. returning a result to the backend

System flow

Every unlock attempt follows the same path from camera to door.



The module stops at the backend decision; everything after it belongs to the circuit side.

1. Scanning the face

The laptop's built-in camera captures live video with OpenCV, and a detection model finds the face in each frame before anything else happens.

1. **Capture** — open the built-in camera with OpenCV and read frames at about 640×480. Low resolution keeps it fast on a laptop CPU.
2. **Detect** — locate the face box and key points (eyes, nose, mouth).
3. **Quality check** — skip frames where the face is too small, blurry, or badly lit, so bad images never reach matching.
4. **Liveness check** — block someone holding up a photo or phone screen. For the demo, a simple blink or head-movement check is enough.
5. **Align** — rotate and crop the face using the eye positions so every face enters the next step in the same pose.

2. Memorizing faces

The system remembers a face as a numeric vector (an embedding), not as a photo, so new people can be added without retraining any model.

Enrollment (registering a person)

1. Admin enters the person's name/ID in the backend.
2. The camera takes 5–10 images with slightly different angles and expressions.
3. Each image goes through detection and alignment (section 1).
4. A pre-trained ArcFace model (InsightFace) turns each face into a 512-number embedding.
5. The embeddings are averaged into one reference vector and saved to the database.

Recognition (at the door)

1. The live face is converted to an embedding the same way.
2. It is compared against every stored vector using cosine similarity.
3. If the best score passes a threshold, the person is identified; otherwise the result is "unknown".
The threshold will be tuned on our own test photos to balance false accepts against false rejects.

Why embeddings instead of training a classifier: a classifier (e.g. a CNN trained on our team's faces) must be retrained every time someone is added and needs many photos per person. Embedding matching needs a few photos and no training. A simpler alternative is the `face_recognition` library (dlib, 128-number vectors), which is easier to set up but less accurate.

Planned tables

Table	Main columns	Purpose
users	id, name, role, is_active, created_at	Registered people (shared with fingerprint and password)
face_embeddings	id, user_id, embedding (BLOB), model_name, created_at	Stored face vectors, linked to a user
access_logs	id, user_id, method, result, score, timestamp	Every attempt: who, which method, granted or denied

Only embeddings are stored, not face photos, which keeps the database small and avoids keeping raw images of people. Matching loads all embeddings into memory at startup, so each check is a fast in-memory comparison.

Tech stack

Part	Stack	Alternative choice
Language	Python	
Camera capture (laptop built-in camera)	OpenCV	
Face detection	InsightFace RetinaFace	OpenCV YuNet, MediaPipe
Face embedding	InsightFace ArcFace (512-d)	face_recognition / dlib (128-d)
Liveness	Blink / head-movement check	Pretrained anti-spoofing model
Database	SQLite	
Backend API	FastAPI	Flask

Risks and fallbacks

Risk	Fallback
Poor lighting at the demo spot	Add a small LED light; enroll faces under the same lighting
Photo or phone-screen spoofing	Liveness check; require face + password for high security
Wrong person accepted	Stricter threshold; test with non-registered faces
Slow on laptop CPU	Use lighter detector (YuNet) and process every 2nd–3rd frame
Glasses, masks, hairstyle changes	Enroll several images per person, including with glasses

Milestones

Process	Deliverable
1	Camera capture + face detection working
2	Embedding generation + matching on test photos
3	Enrollment flow + SQLite tables
4	Liveness check + threshold tuning
5	Connected to team backend and circuit
6	Full door demo test and fixes